CSS Specificity

What Is Specificity?

If two CSS selectors apply to the same element, the one with higher specificity wins.

Specificity Hierarchy

Every selector has its place in the specificity hierarchy. There are four distinct categories which define the specificity level of a given selector:

1. Inline styles (Presence of style in document). An inline style lives within your HTML document. It is attached directly to the element to be styled. E.g. `<h1 style="color: #fff;">`
2. IDs (# of ID selectors) ID is an identifier for your page elements, such as `#div`.
3. Classes, attributes and pseudo-classes (# of class selectors). This group includes `.classes`, [attributes] and pseudo-classes such as `:hover`, `:focus` etc.
4. Elements and pseudo-elements (# of Element (type) selectors). Including for instance `:before` and `:after`.

How To Measure Specificity?

style attribute ie. style=""	1000
ID	100
attribute, class or pseudo-class	10
element name or pseudo-element	1

So for this rule:

```
body #content .data img:hover
```

the specificity value would be 122 (**0,1,2,2** or 0122): 100 for #content, 10 for .data, 10 for :hover, 1 for body and 1 for img." [CSS Specificity]

Alternative way: "Count the number of ID attributes in the selector (= a). Count the number of other attributes and pseudo-classes in the selector (= b). Count the number of element names and pseudo-elements in the selector (= c). Concatenating the three numbers **a-b-c gives the specificity**.

Specificity Examples: Test Yourself

It's easier to calculate the specificity using the first method. Let's find out, how it actually is done.

1	* { }	0
2	li { }	1 (one element)
3	li:first-line { }	2 (one element, one pseudo-element)
4	ul li { }	2 (two elements)
5	ul ol+li { }	3 (three elements)
6	h1 + *[rel=up] { }	11 (one attribute, one element)
7	ul ol li.red { }	13 (one class, three elements)
8	li.red.level { }	21 (two classes, one element)
9	style=""	1000 (one inline styling)
10	p { }	1 (one HTML selector)
11	div p { }	2 (two HTML selectors)
12	.sith	10 (one class selector)
13	div p.sith { }	12 (two HTML selectors and a class selector)
14	#sith	100 (one id selector)
15	body #darkside .sith p { }	112 (HTML selector, id selector, class selector, HTML selector; 1+100+10+1)

Specificity: Basic Principles

Equal specificity: the latest rule is the one that counts. "If you have written the same rule into your external style sheet twice, then the lower rule in your style sheet is closer to the element to be styled, it is deemed to be more specific and therefore will be applied. When selectors have an equal specificity value, such as

```
#content h1 {
padding: 5px;
}

#content h1 {
padding: 10px;
}
```

where both rules have the specificity **0, 1, 0, 1**, the latter rule is always applied.

Specificity Rules

ID selectors have a higher specificity than attribute selectors. For example, in HTML, the selector `#p123` is more specific than `[id=p123]` in terms of the cascade. Example: in

```
A:
a#a-02 { background-image : url(n.gif); }

and

B:
a[id="a-02"] { background-image : url(n.png); }
```

the first rule (A) is more specific than the second one (B).

Contextual selectors are more specific than a single element selector. It also holds for other selectors involving more than one HTML element selector.

The embedded style sheet is closer to the element to be styled. So in the following situation

CSS:

```
#content h1 {
padding: 5px;
}
```

HTML:

```
<style type="text/css">
#content h1 {
padding: 10px;
}
</style>
```

the latter rule will be applied.

The last rule defined overrides any previous, conflicting rules. For example, given these two rules

```
p { color: red; background: yellow }
p { color: green }
```

…paragraphs would appear in green text. They would also have a yellow background, however, because the first rule is not completely negated.

A class selector beats any number of element selectors.
`.introduction` beats `html body div div h2 p`.